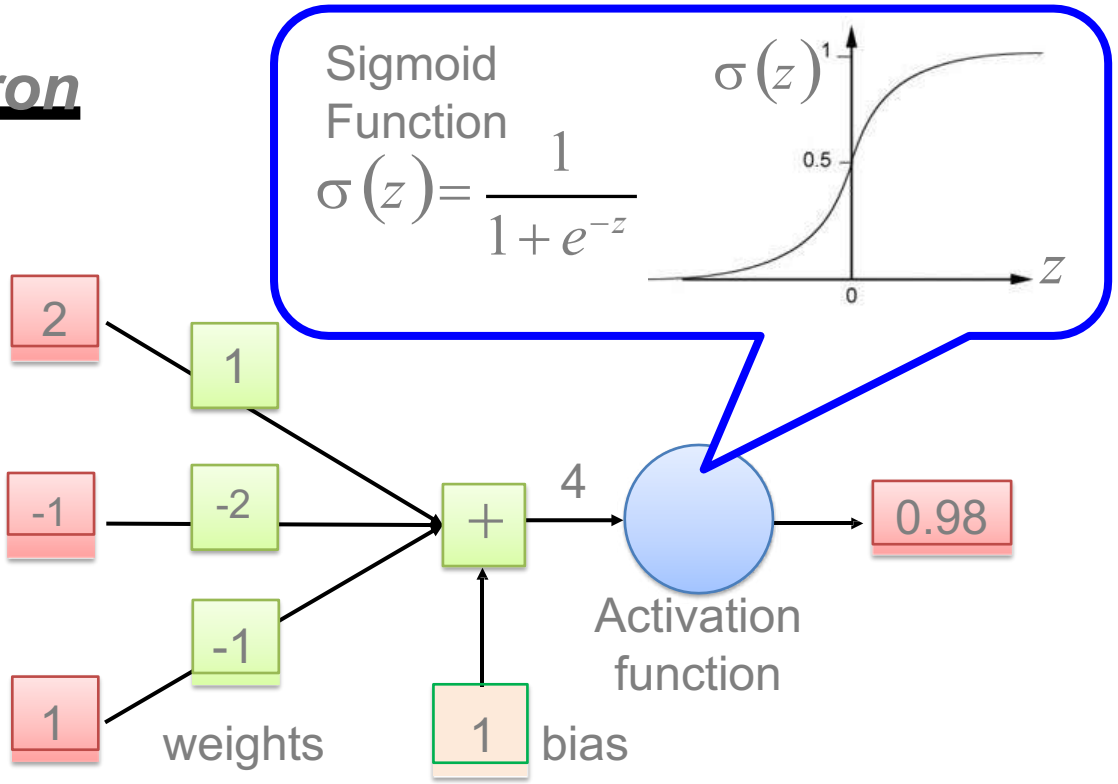
LECTURE 04 ANN

Neural Network

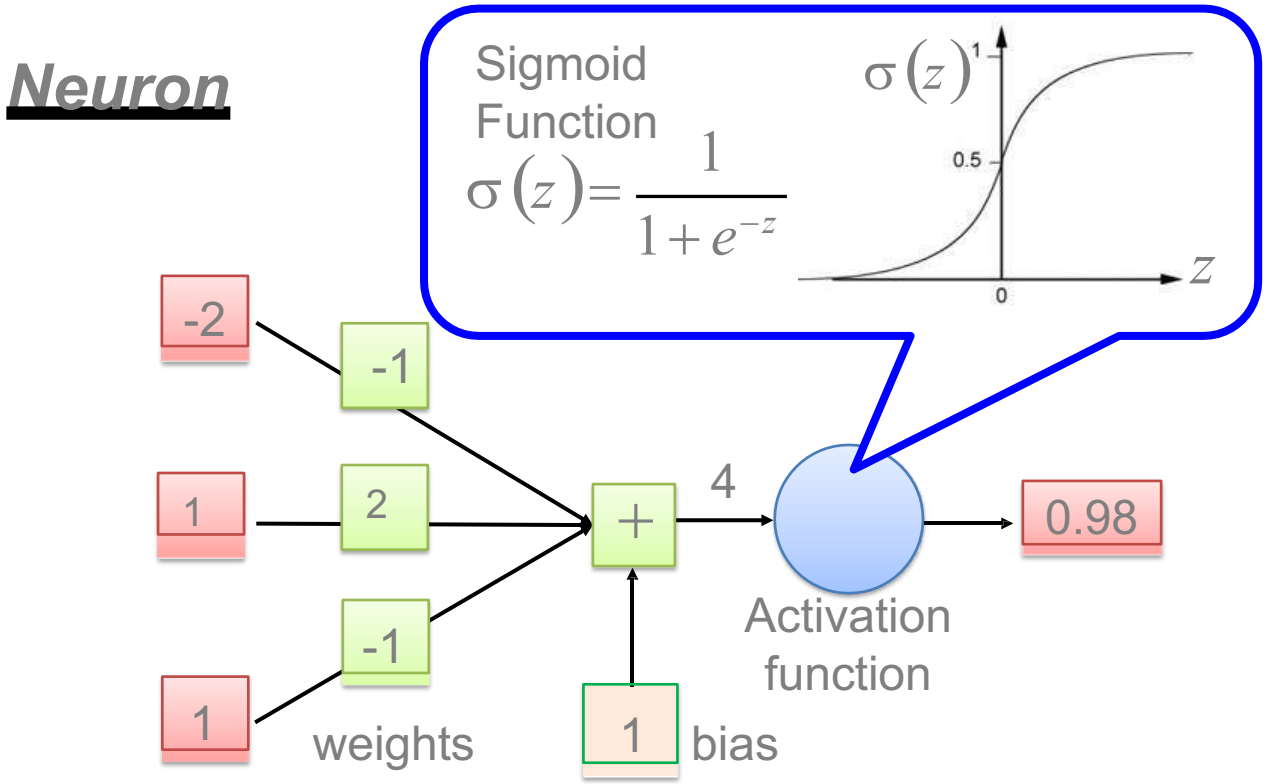
$$z = a_1 w_1 + a_k w_k \ + \ldots + a_K w_K + b$$

Neuron



Neural Network

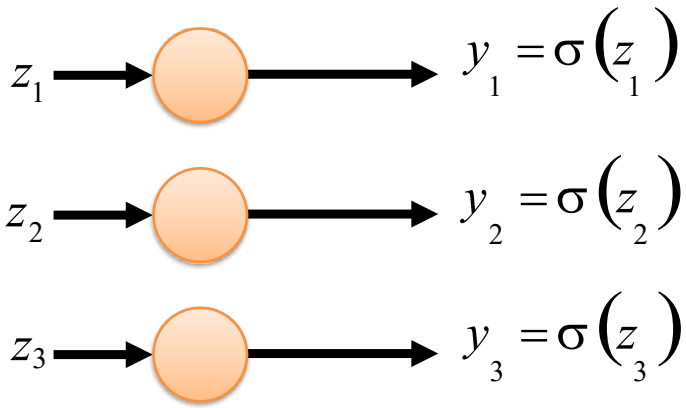
$$z = a_1 w_1 + a_k w_k \ + \ldots + a_K w_K + b$$



Output Layer

- Softmax layer as the output layer

Ordinary Layer



In general, the output of network can be any value.

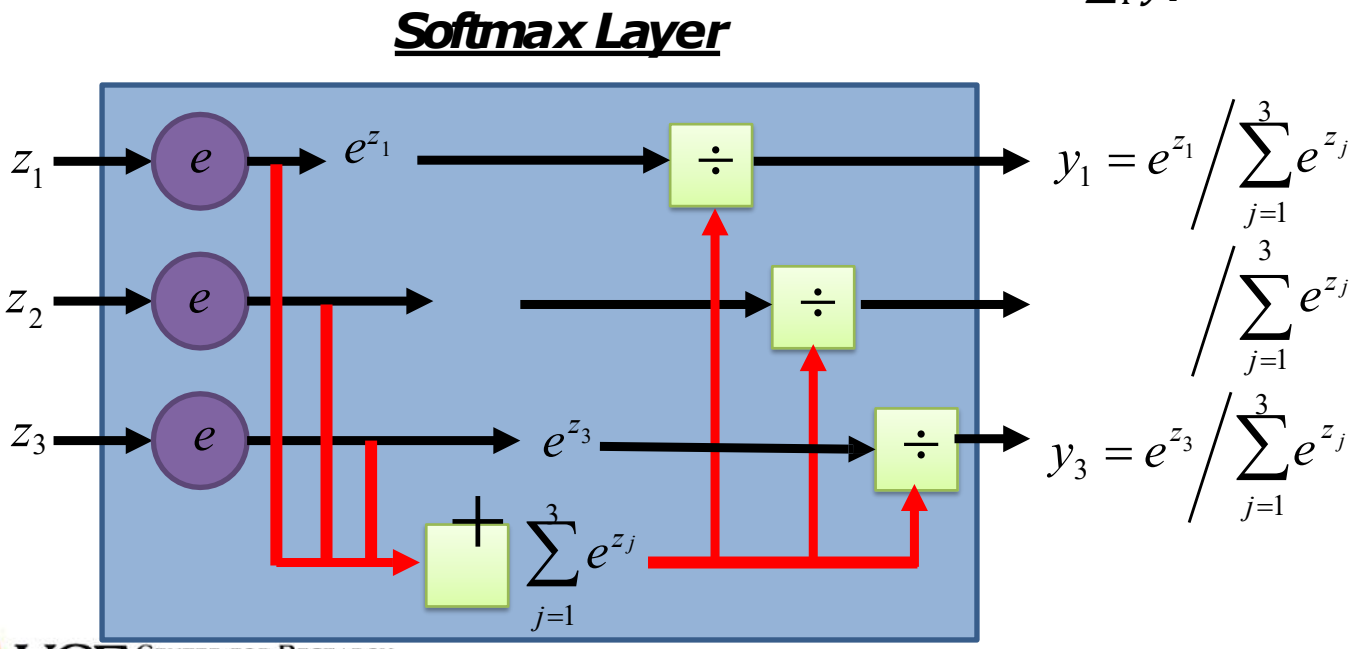
May not be easy to interpret

Output Layer

- Softmax layer as the output layer

Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$



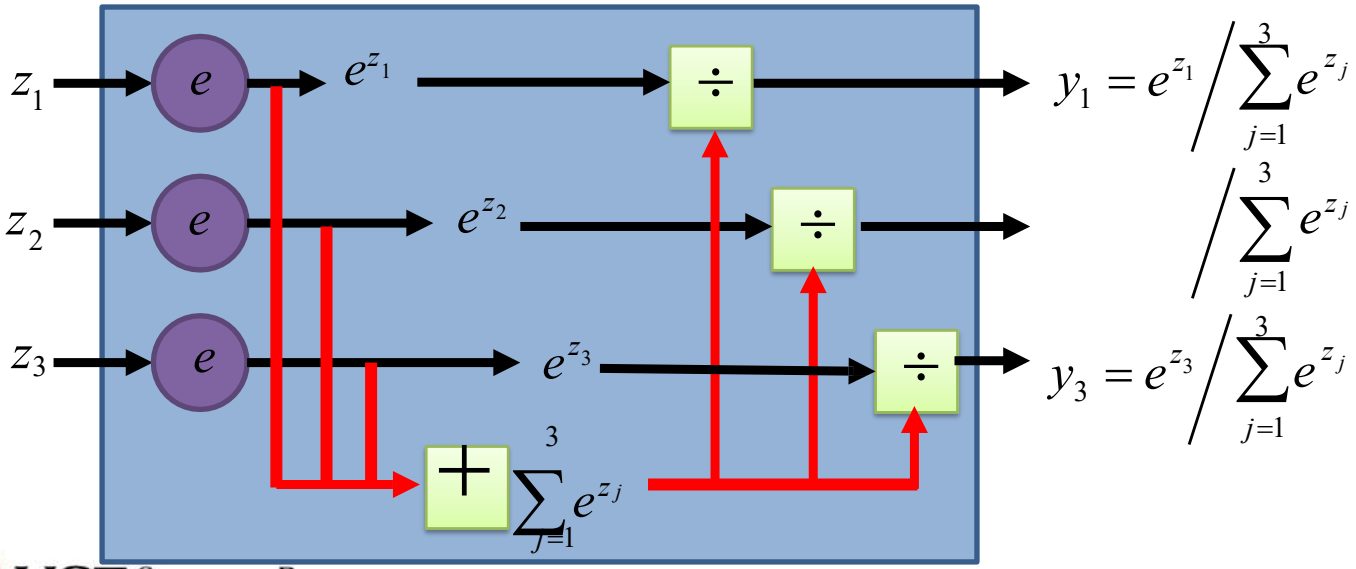
Output Layer

- Softmax layer as the output layer

Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$

Softmax Layer

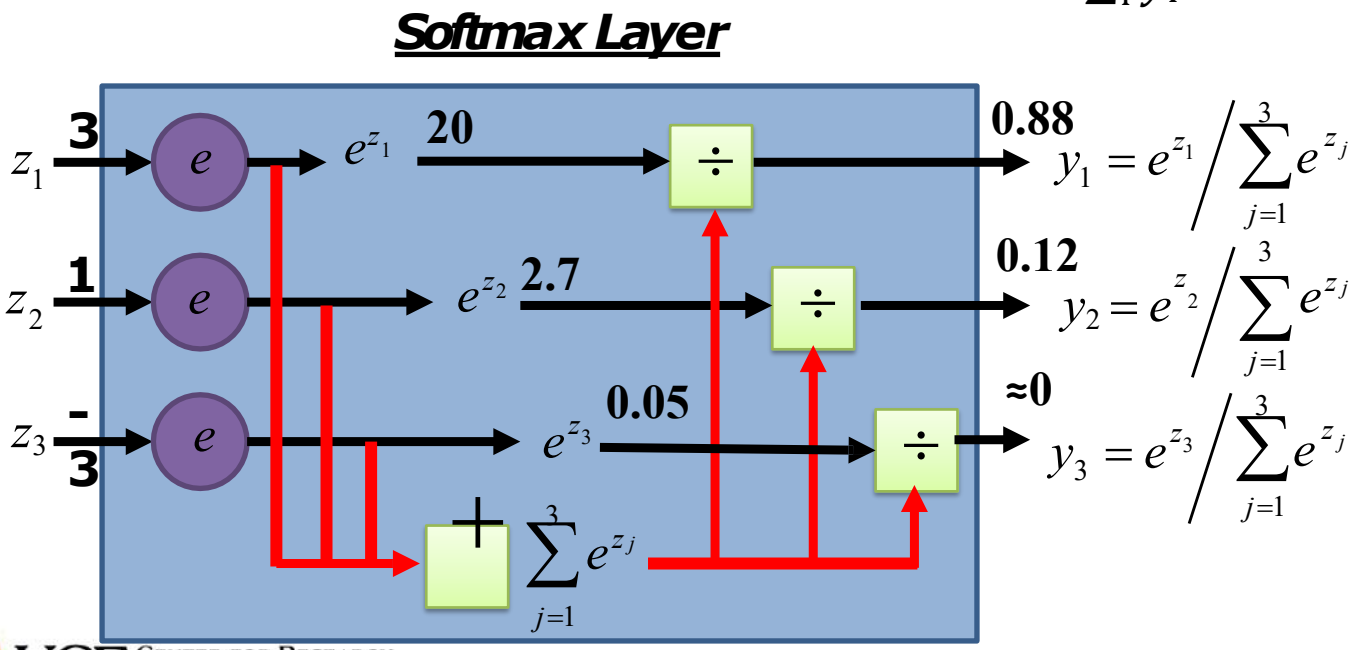


Output Layer

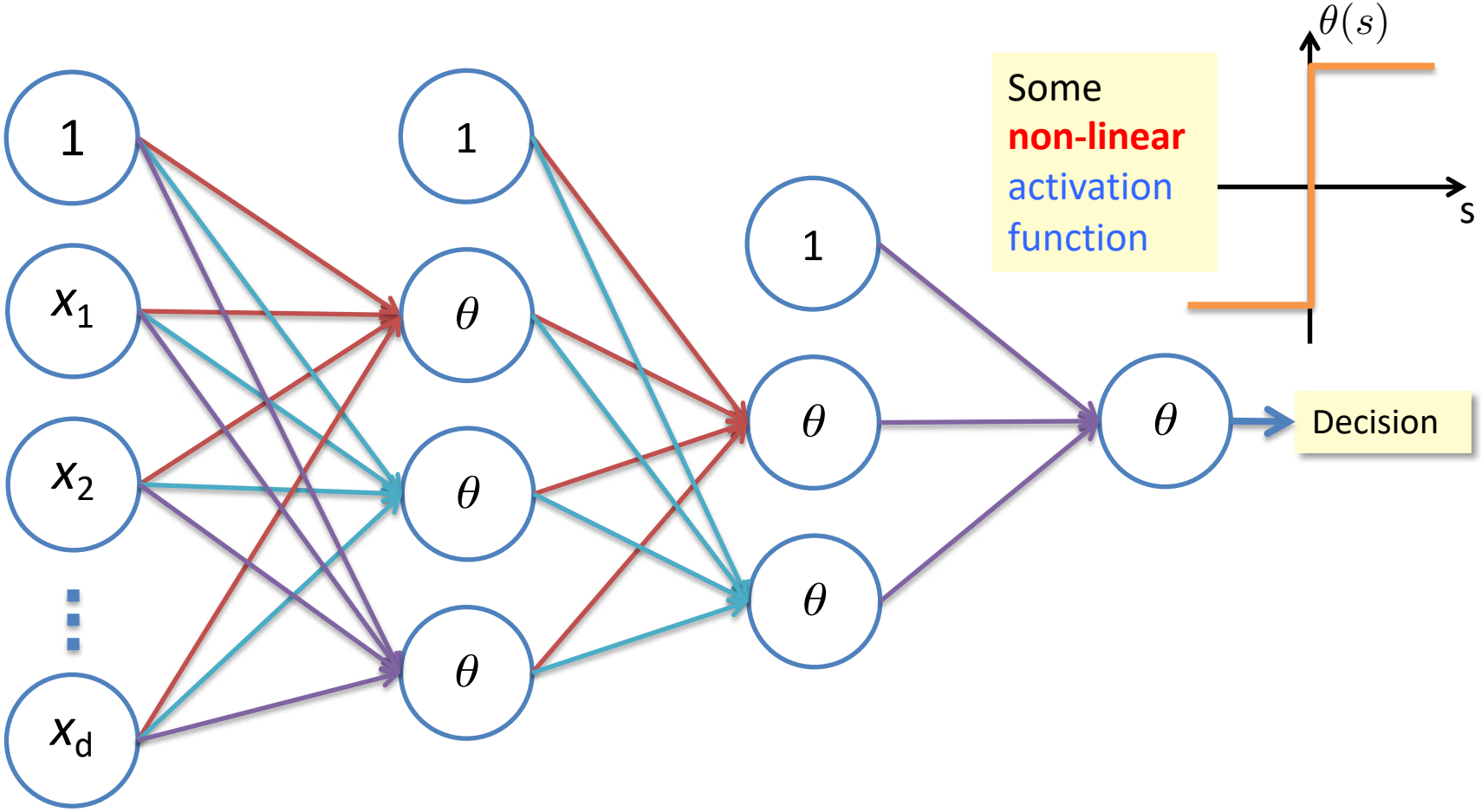
- Softmax layer as the output layer

Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$



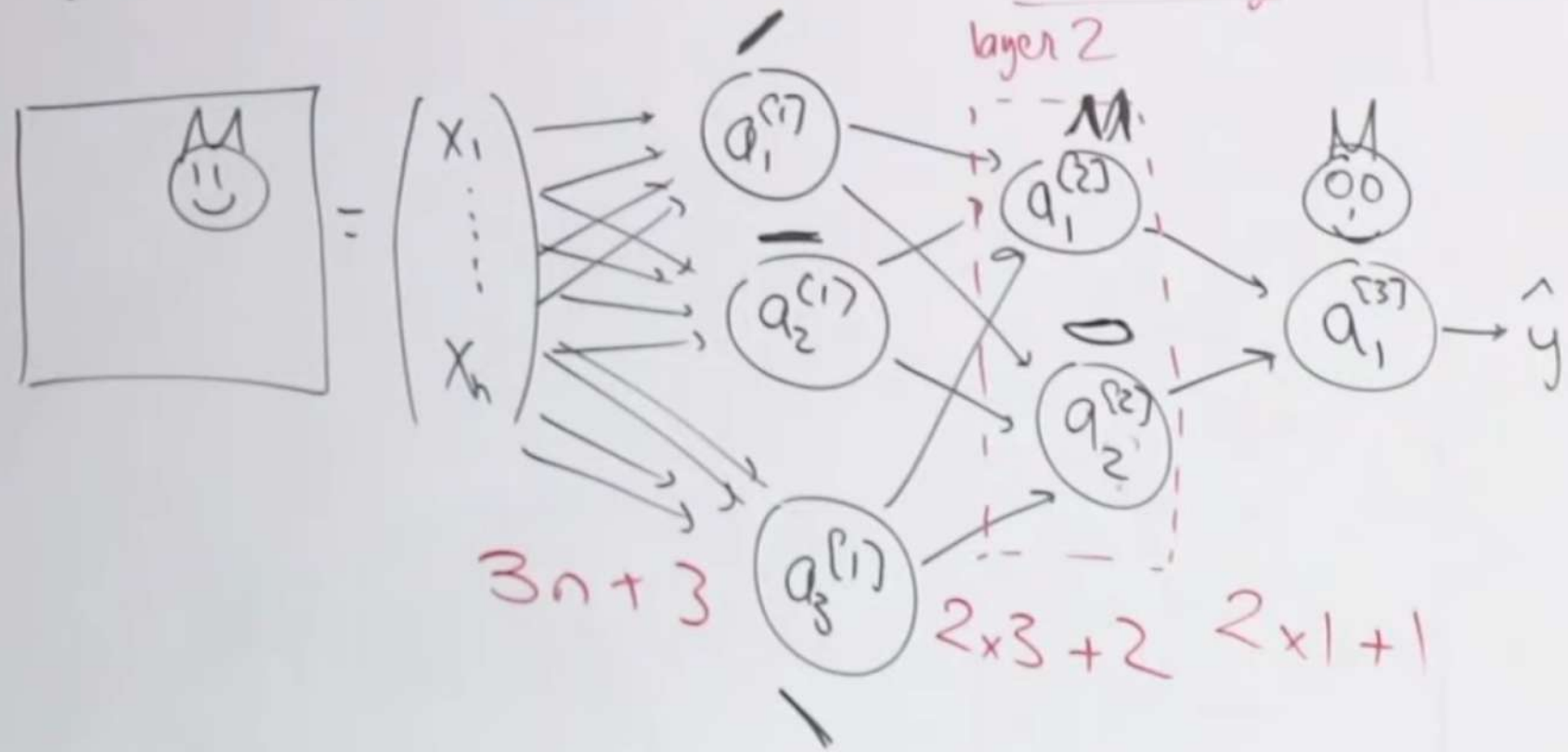
Feed-forward Neural Network



input x hidden layers $1 \le l \le L$ output layer $l = L$

II Neural Networks

goal image ⇒ cat vs. no cat
(1) (0)



Propagation equations

$$z^{[1]} = W^{[1]} x + b^{[1]}$$

$(3,n)$ $(n,1)$ $(3,1)$

$$a^{[1]} = \sigma(z^{[1]})$$

$(3,1)$ $(3,1)$

$$z^{[2]} = W^{[2]} a^{[1]} + b^{[2]}$$

$(2,3)$ $(3,1)$ $(2,1)$

$$a^{[2]} = \sigma(z^{[2]})$$

$(2,1)$ $(2,1)$

$$z^{[3]} = W^{[3]} a^{[2]} + b^{[3]}$$

$(1,2)$ $(2,1)$ $(1,1)$

$$a^{[3]} = \sigma(z^{[3]})$$

$(1,1)$ $(1,1)$

* What happens for an input batch of m examples?

$$X = \begin{pmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{pmatrix}$$

idx of ex

Broadcasting

$$Z^{[1]} = W^{[1]} X + b^{[1]}$$

$(3,n)$ (n,m) $(3,1)$

$(3,m)$

$$= \begin{bmatrix} | & & | \\ z^{[1](1)} & \cdots & z^{[1](m)} \\ | & & | \end{bmatrix}$$

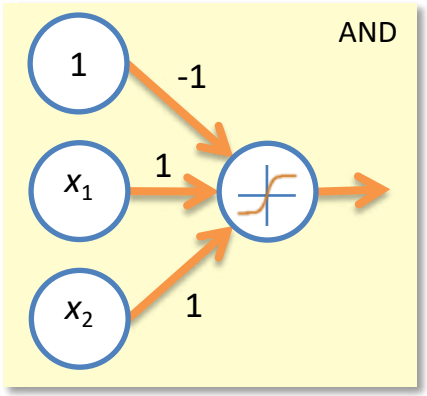
$$\tilde{b}^{[1]} = \begin{pmatrix} | & | & & | \\ b^{[1]} & b^{[1]} & \cdots & b^{[1]} \\ | & | & & | \end{pmatrix}$$

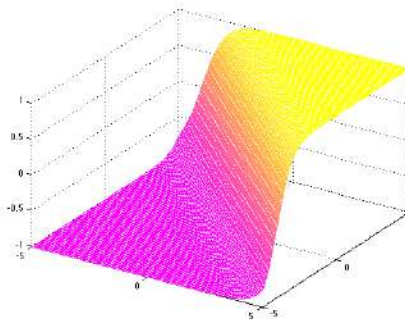
m

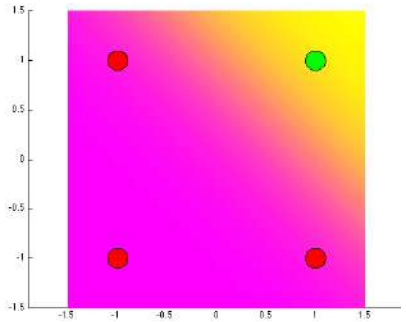
Activation Function

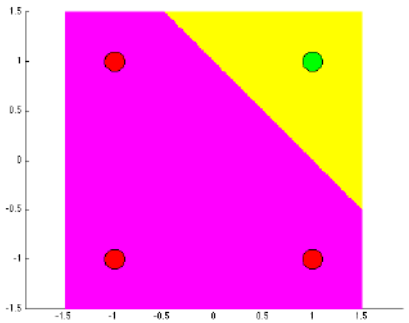
$$\tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

- A **non-linear** activation function must be used to allow flexibility of decision boundary
- Example activation functions:
  $2U(s) - 1$
  $\tanh(s)$
  …
- Example





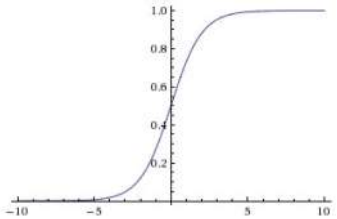




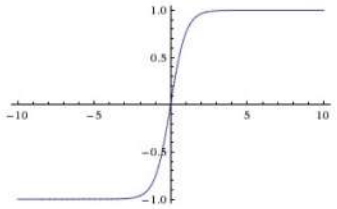
Activation Functions

Sigmoid

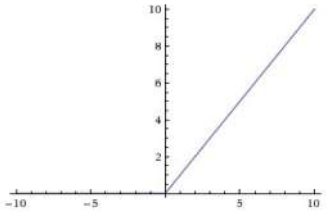
$$\sigma(x) = 1/(1+e^{-x})$$



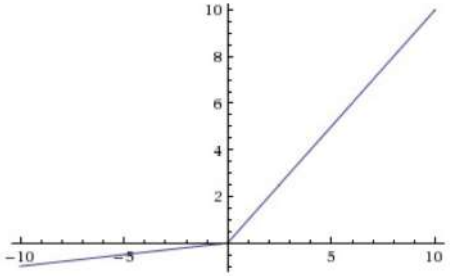
tanh tanh(x)



ReLU max(0,x)



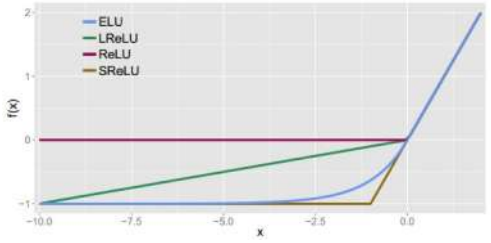
Leaky ReLU
max(0.1x, x)



Maxout $\max(w_1^T x + b_1, w_2^T x + b_2)$

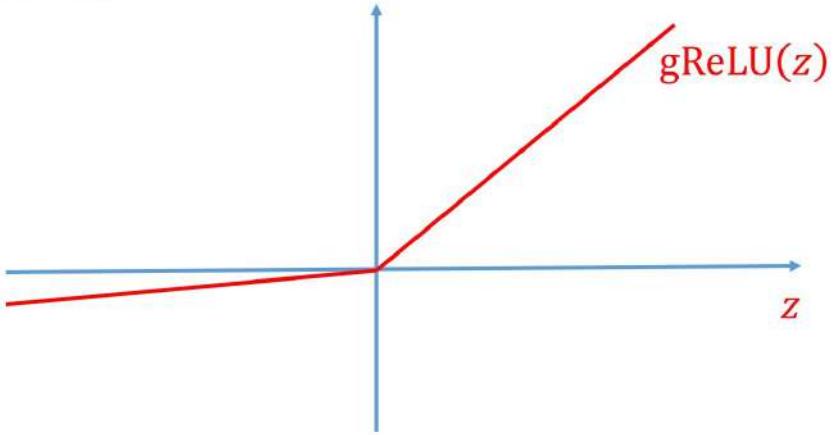
ELU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$



Activation Function

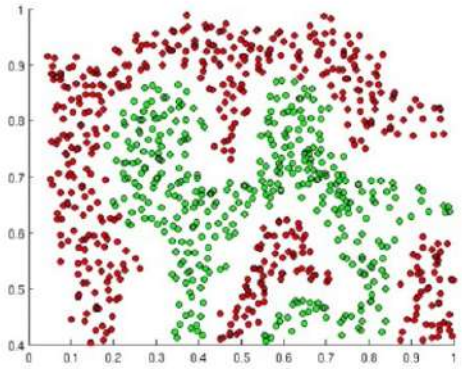
- Generalizations of ReLU $\mathrm{gReLU}(z) = \max\{z, 0\} + \alpha \min\{z, 0\}$
  - Leaky-ReLU$(z) = \max\{z, 0\} + 0.01 \min\{z, 0\}$
  - Parametric-ReLU(z): α learnable



Activation Function

- Why non-linear activation function?

The purpose of activation functions is to ***introduce non-linearities*** *into the network*

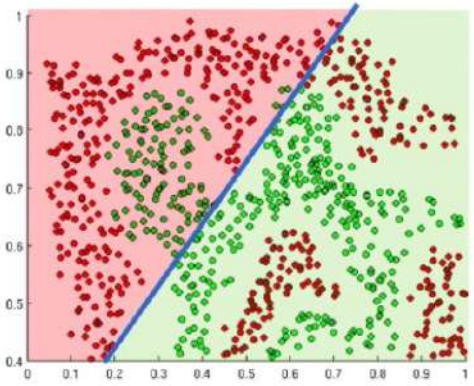


What if we wanted to build a Neural Network to distinguish green vs red points?

Activation Function

- Why non-linear activation function?

The purpose of activation functions is to ***introduce non-linearities*** *into the network*

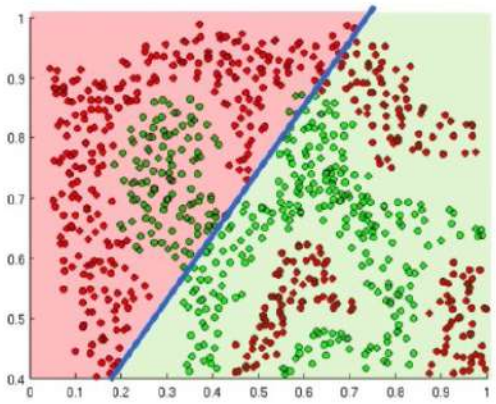


Linear Activation functions produce linear decisions no matter the network size

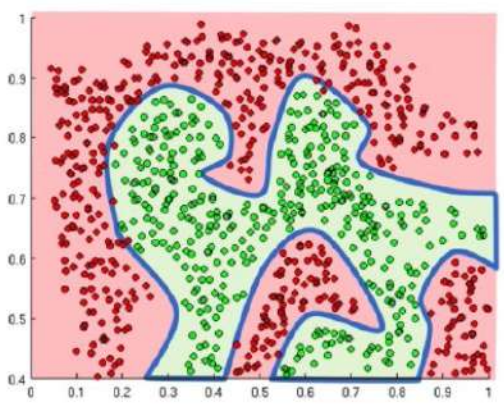
Activation Function

- Why non-linear activation function?

*The purpose of activation functions is to **introduce non-linearities** into the network*



Linear Activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Activation Function

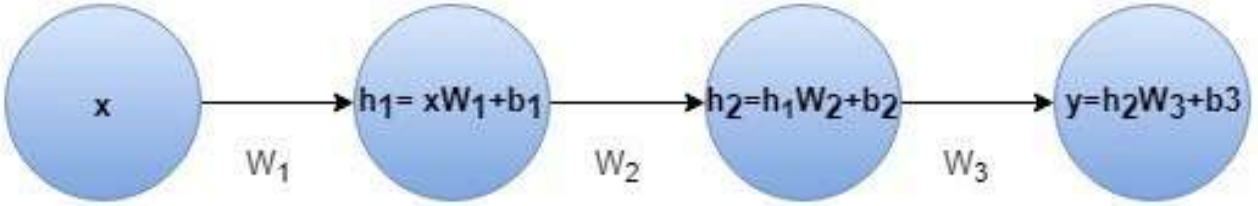
- Why non-linear activation function?

- **All layers of the neural network collapse into one**—with linear activation functions, no matter how many layers in the neural network, the last layer will be a linear function of the first layer (because a linear combination of linear functions is still a linear function). So a linear activation function turns the neural network into just one layer.

- A neural network with a linear activation function is simply a linear regression model. It has limited power and ability to handle complexity varying parameters of input data.

Activation Function

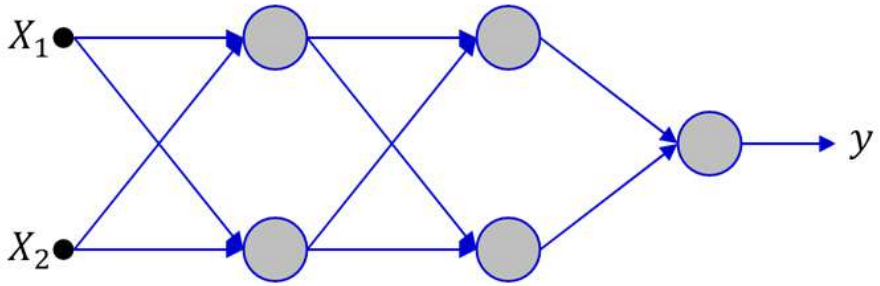
- Why non-linear activation function?




```
y = h2 * W3 + b3
  = (h1 * W2 + b2) * W3 + b3
  = h1 * W2 * W3 + b2 * W3 + b3
  = (x * W1 + b1) * W2 * W3 + b2 * W3 + b3
  = x * W1 * W2 * W3 + b1 * W2 * W3 + b2 * W3 + b3
  = x * W' + b'
```

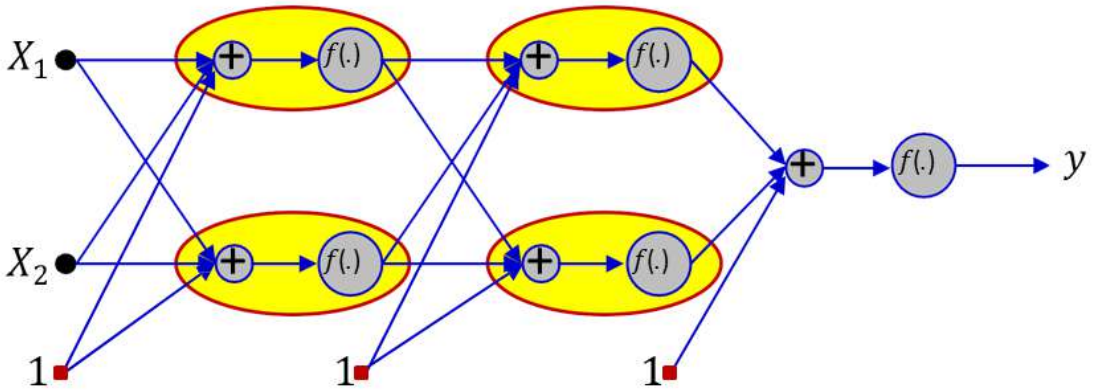
A linear activation function turns the neural network into just one layer.

A first closer look at the network



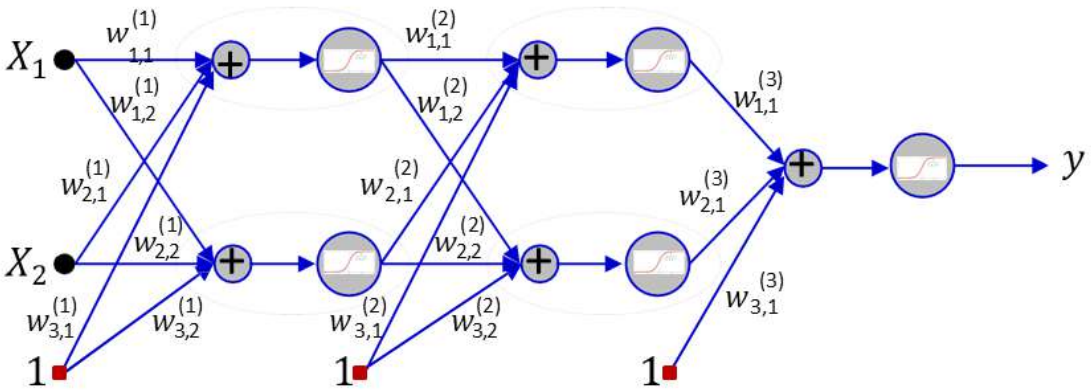
- Showing a tiny 2-input network for illustration
  - Actual network would have many more neurons and inputs

A first closer look at the network



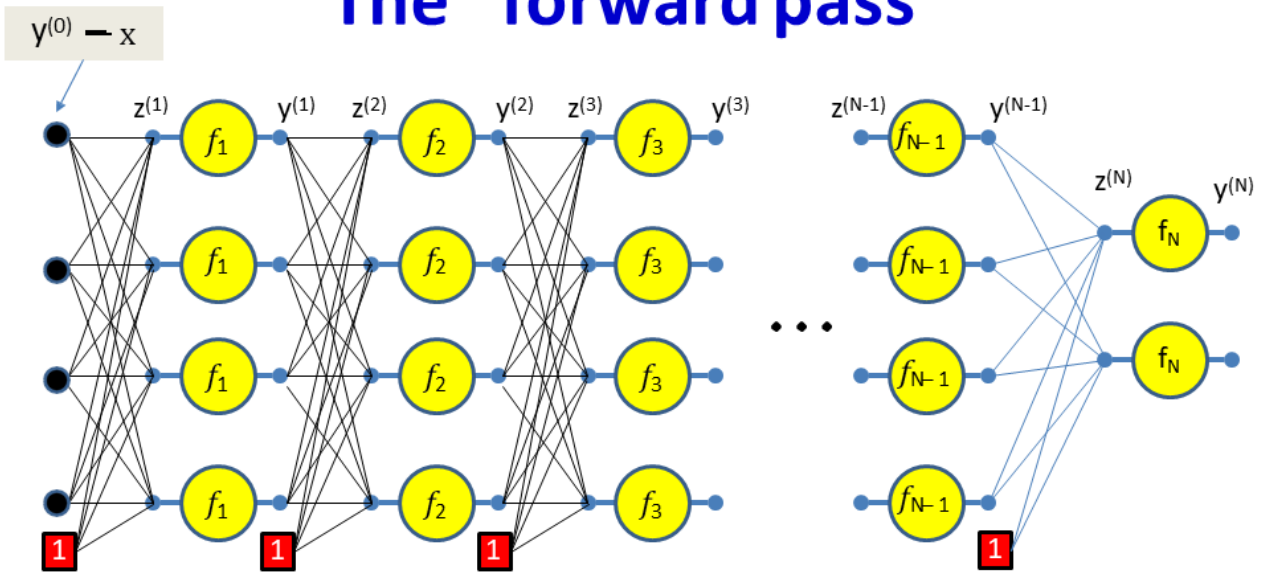
- Showing a tiny 2-input network for illustration
  - Actual network would have many more neurons and inputs
- Explicitly separating the weighted sum of inputs from the activation

A first closer look at the network



- Showing a tiny 2-input network for illustration
  - Actual network would have many more neurons and inputs
- Expanded **with all weights shown**
- Let's label the other variables too…

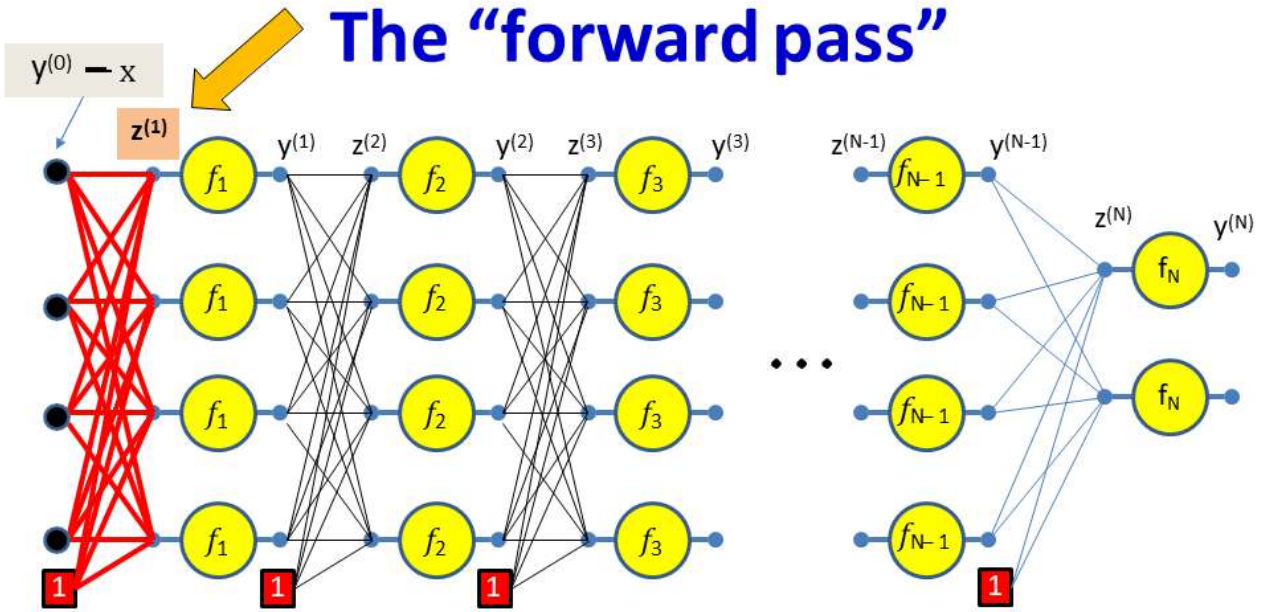
The "forward pass"



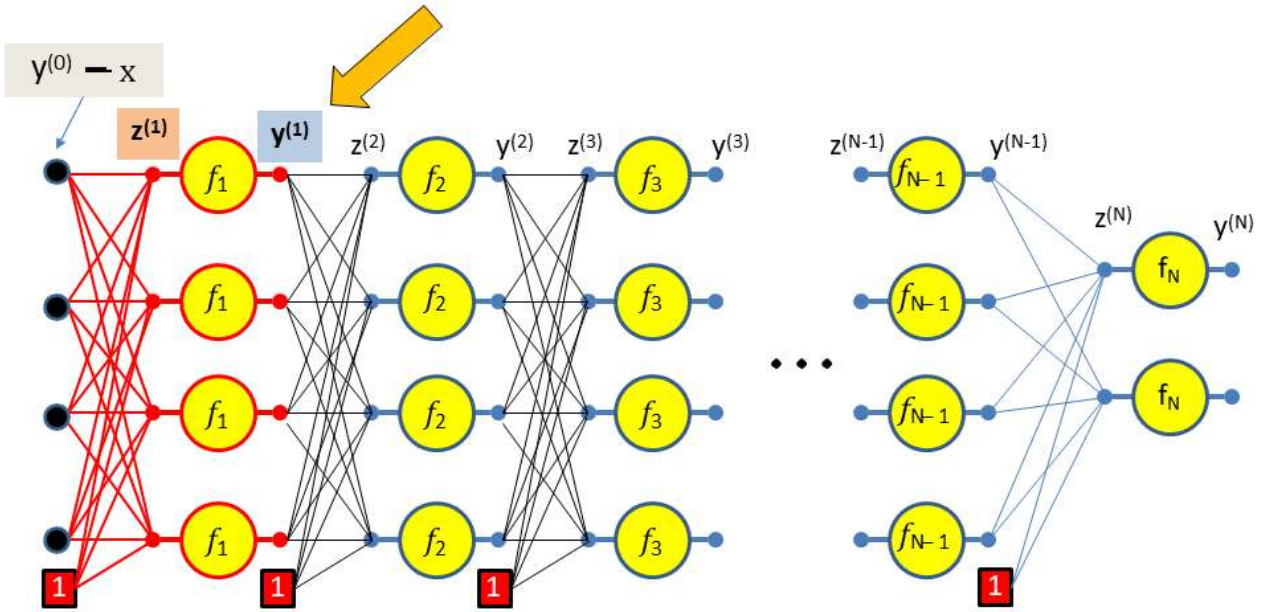
We will refer to the process of computing the output from an input as the *forward pass*

We will illustrate the forward pass in the following slides

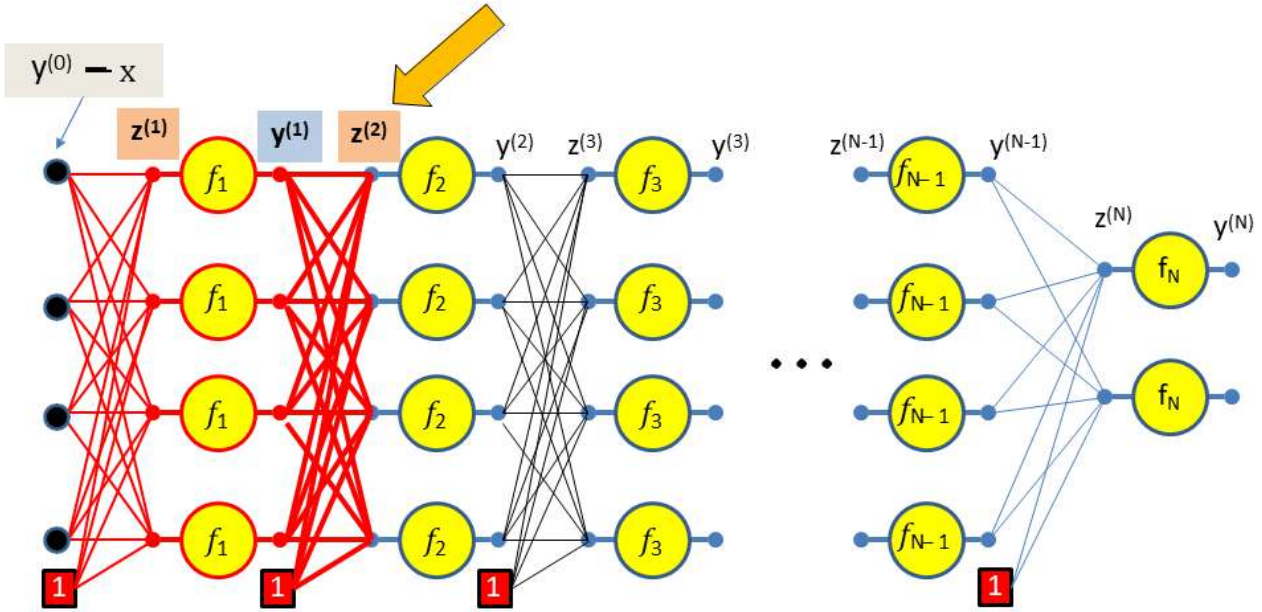
The "forward pass"



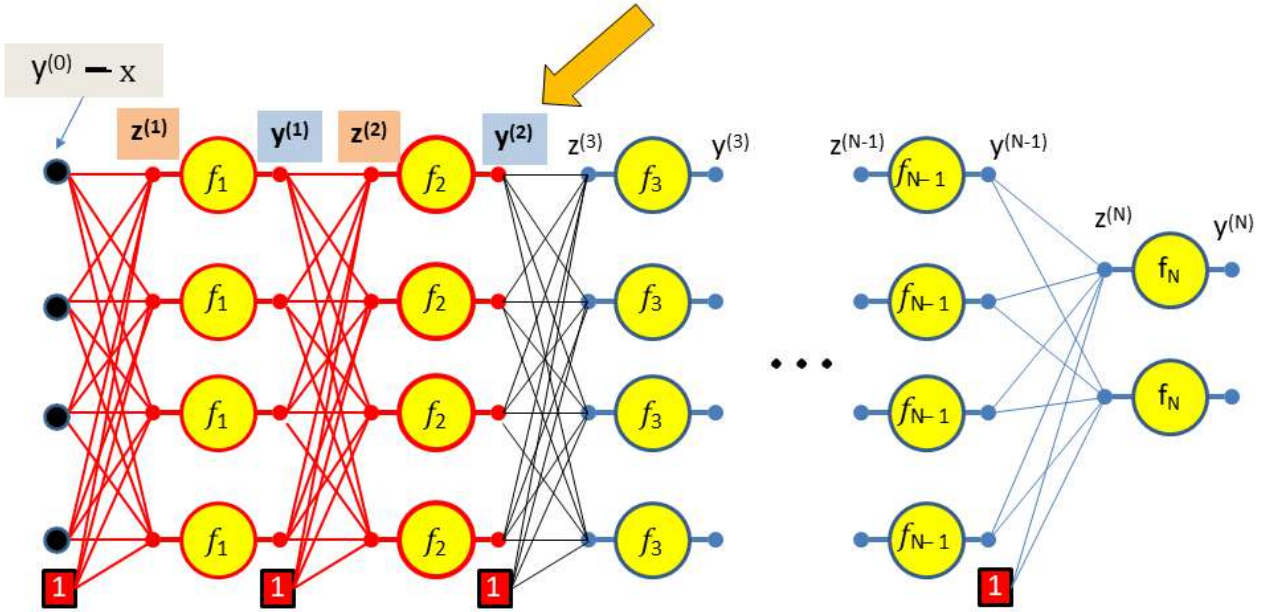
$$z_j^{(1)} = \sum_i w_{ij}^{(1)} \, y_i^{(0)}$$



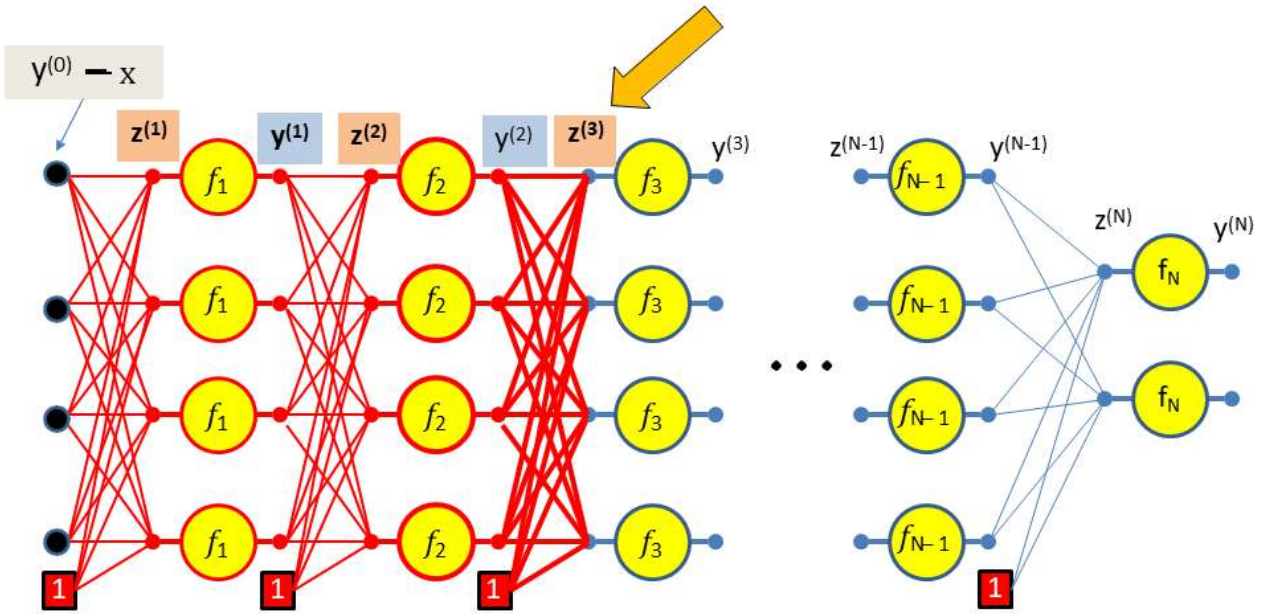
$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)} \qquad y_j^{(1)} = f_1\left(z_j^{(1)}\right)$$



$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)} \qquad y_j^{(1)} = f_1\left(z_j^{(1)}\right) \qquad z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)}$$

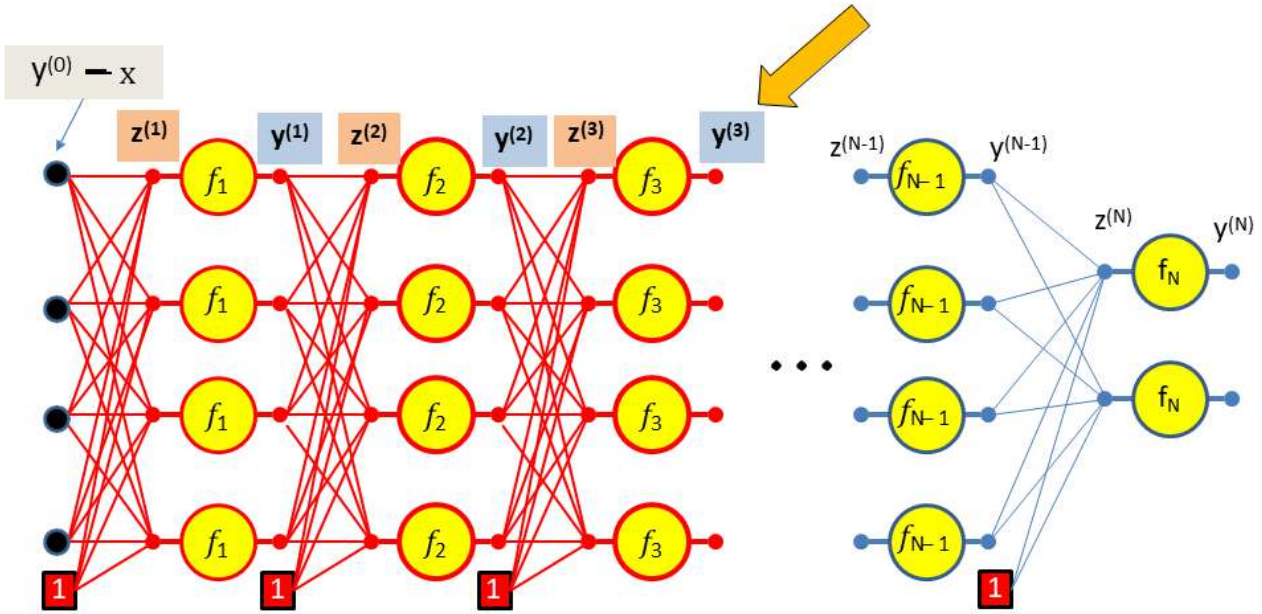


$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)} \qquad y_j^{(1)} = f_1\left(z_j^{(1)}\right) \qquad z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)} \qquad y_j^{(2)} = f_2\left(z_j^{(2)}\right)$$



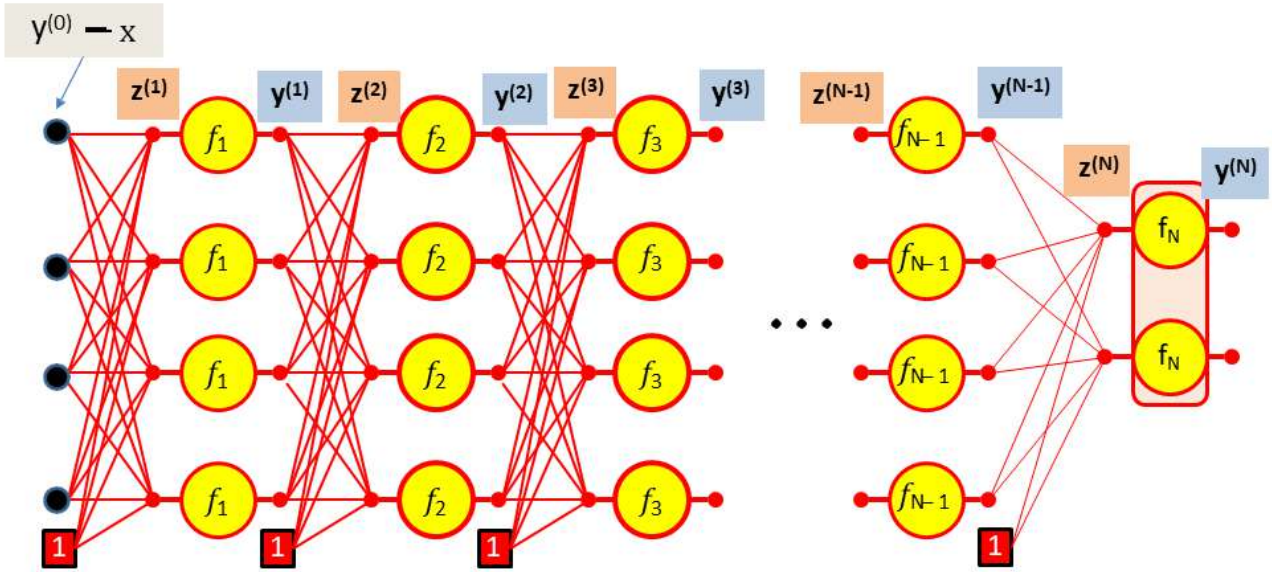
$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)} \qquad y_j^{(1)} = f_1\left(z_j^{(1)}\right) \qquad z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)} \qquad y_j^{(2)} = f_2\left(z_j^{(2)}\right)$$

$$z_j^{(3)} = \sum_i w_{ij}^{(3)} y_i^{(2)}$$



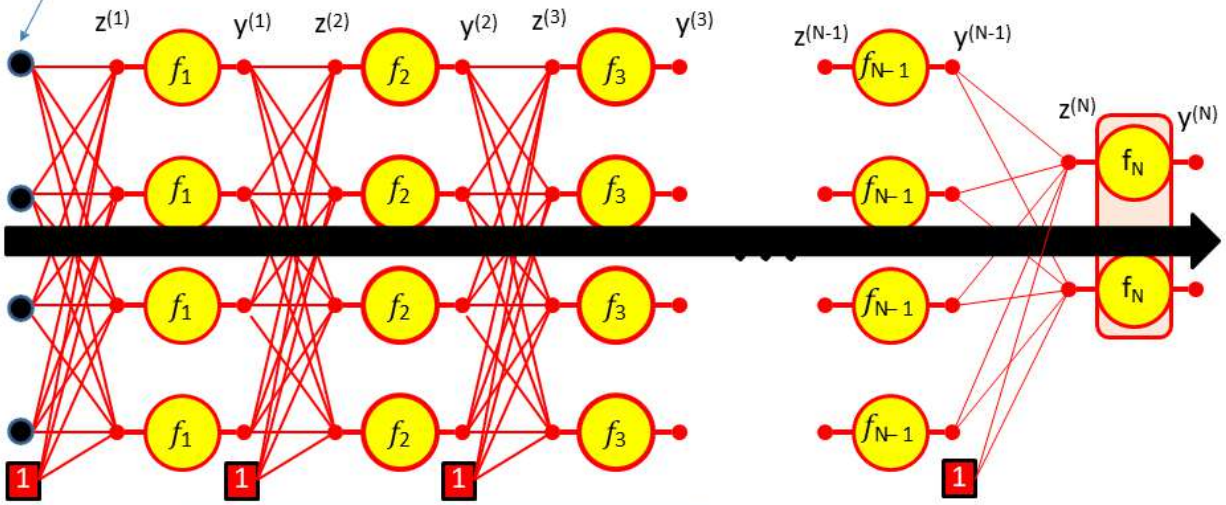
$$z_j^{(1)} = \sum_i w_{ij}^{(1)} y_i^{(0)} \qquad y_j^{(1)} = f_1\left(z_j^{(1)}\right) \qquad z_j^{(2)} = \sum_i w_{ij}^{(2)} y_i^{(1)} \qquad y_j^{(2)} = f_2\left(z_j^{(2)}\right)$$

$$z_j^{(3)} = \sum_i w_{ij}^{(3)} y_i^{(2)} \qquad y_j^{(3)} = f_3\left(z_j^{(3)}\right) \qquad \cdots$$



$$y_j^{(N-1)} = f_{N-1}\left(z_j^{(N-1)}\right) \qquad z_j^{(N)} = \sum_i w_{ij}^{(N)}\, y_i^{(N-1)} \qquad \boldsymbol{y}^{(N)} = f_N\left(\boldsymbol{z}^{(N)}\right)$$

Forward Computation



ITERATE FOR k = 1:N

for j = 1:layer-width

$$y_i^{(0)} = x_i$$

$$z_j^{(k)} = \sum_i w_{ij}^{(k)} y_i^{(k-1)}$$

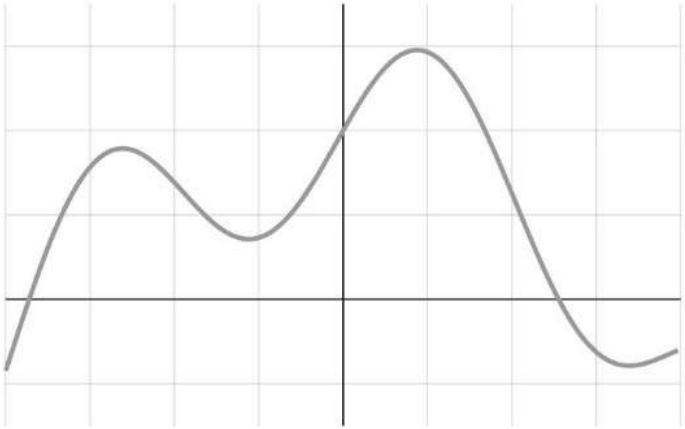
$$y_j^{(k)} = f_k\left(z_j^{(k)}\right)$$

Forward "Pass"

- Input: D dimensional vector $\mathbf{x} = [x_j,\ j = 1 \ldots D]$
- Set:
  - $D_0 = D$, is the width of the 0th (input) layer
  - $y_j^{(0)} = x_j,\ j = 1 \ldots D;$ $\quad y_0^{(k=1\ldots N)} = x_0 = 1$
- For layer $k = 1 \ldots N$
  - For $j = 1 \ldots D_k$ D_k is the size of the kth layer
    - $z_j^{(k)} = \sum_{i=0}^{D_{k-1}} w_{i,j}^{(k)}\ y_i^{(k-1)}$
    - $y_j^{(k)} = f_k\left(z_j^{(k)}\right)$
- Output:
  - $Y = y_j^{(N)}, j = 1..D_N$

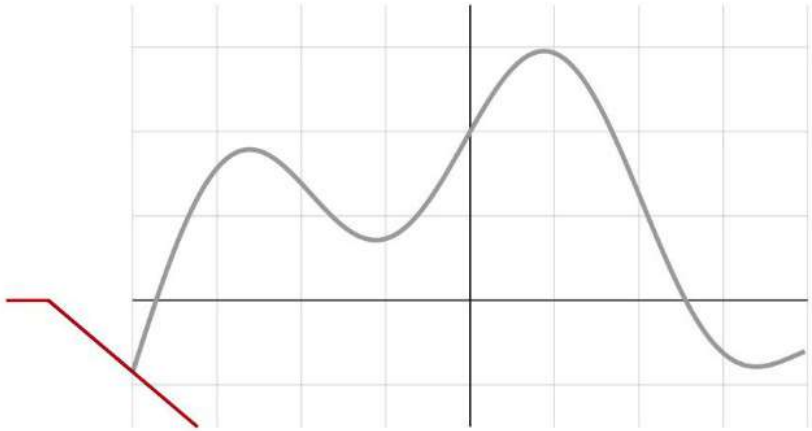
Universal approximation

We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.



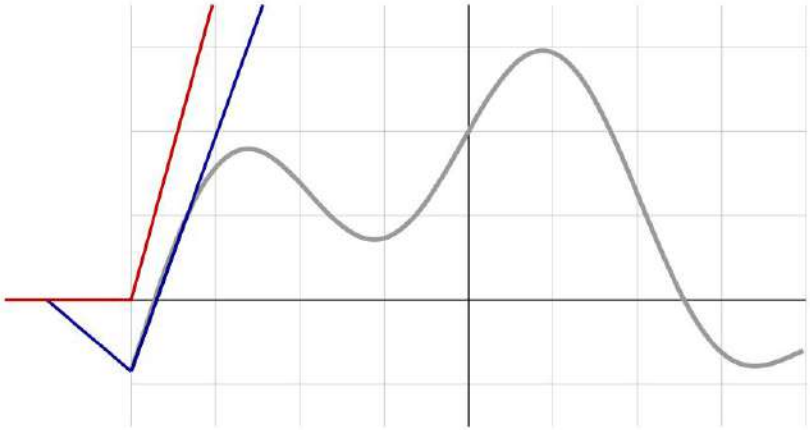
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1)$$



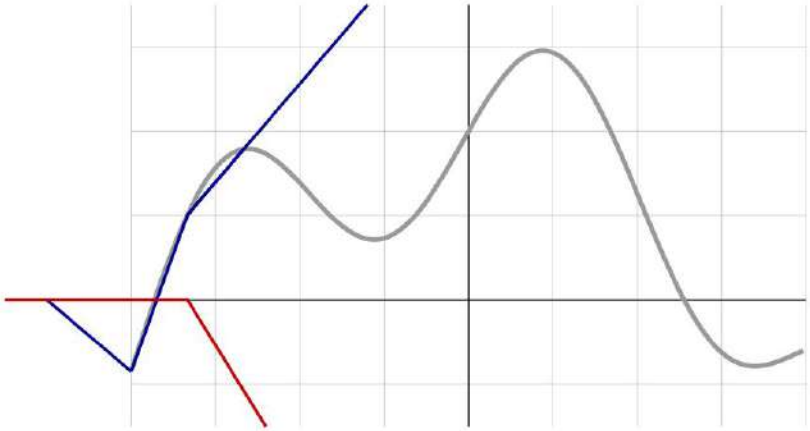
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2)$$



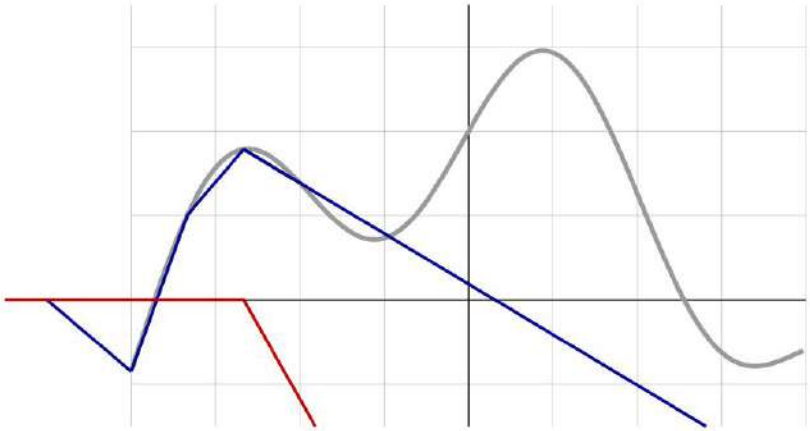
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3)$$



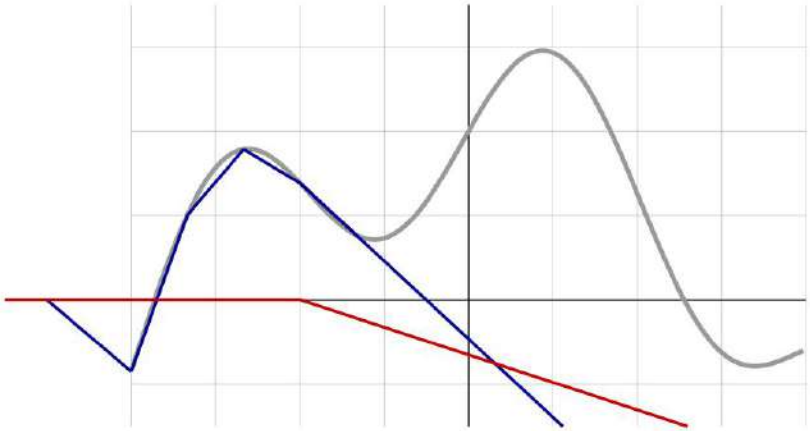
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



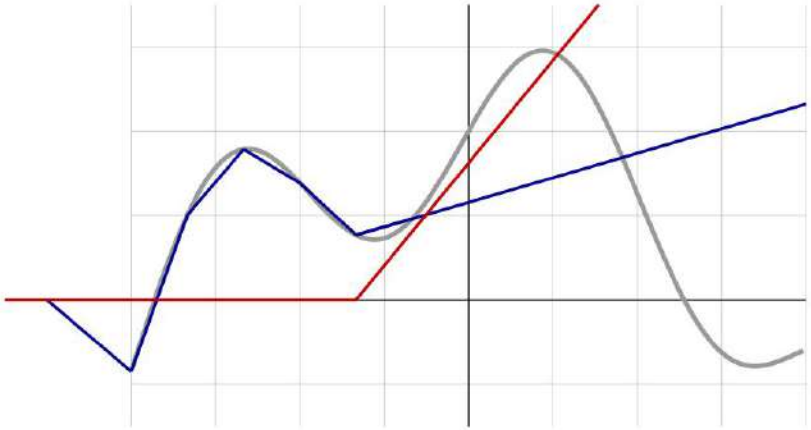
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



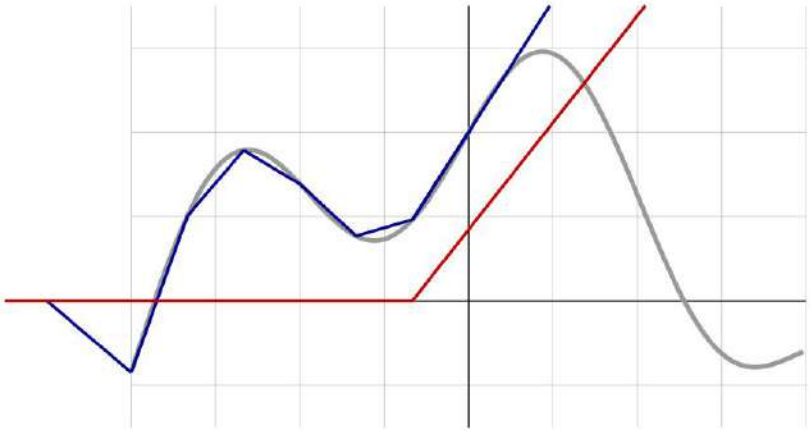
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



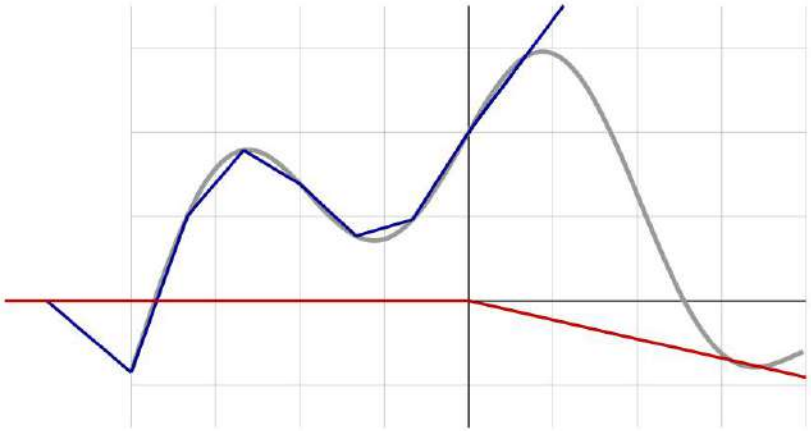
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



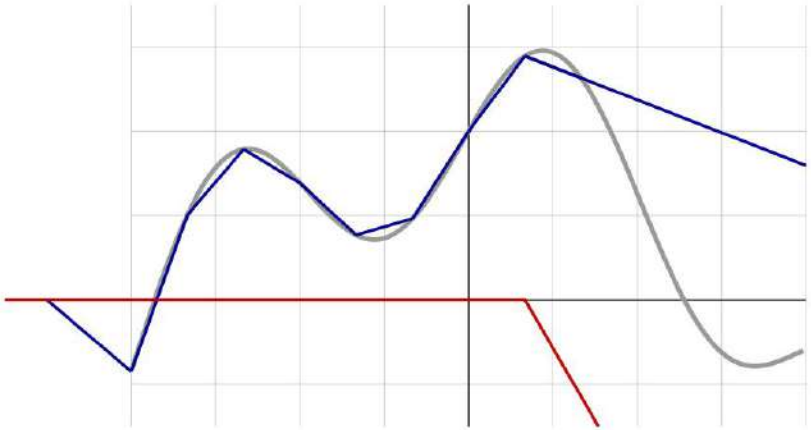
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



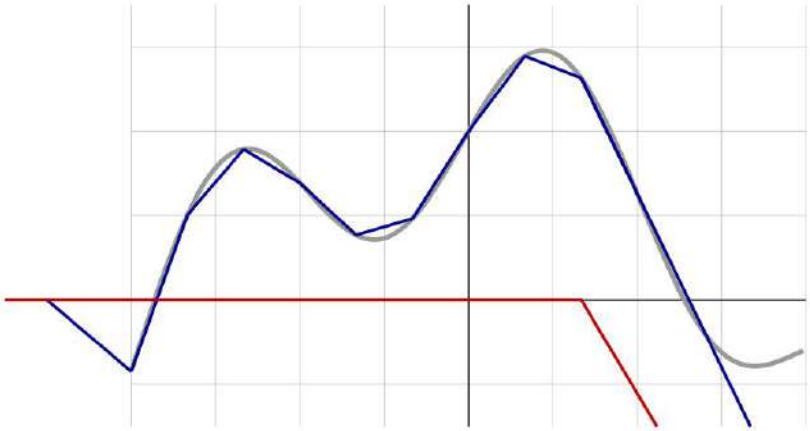
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



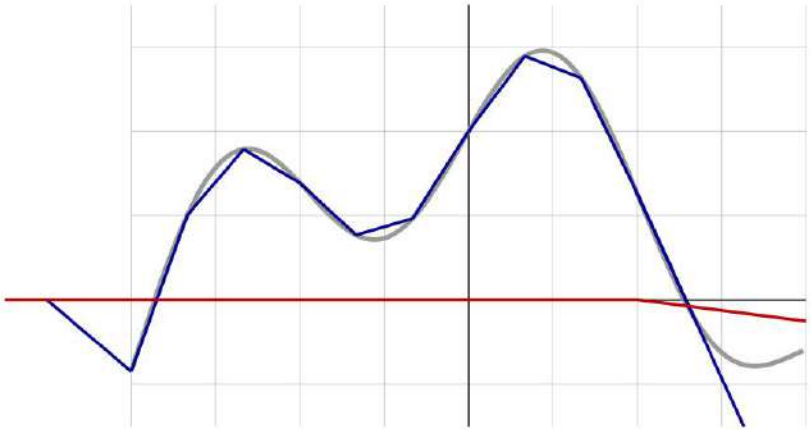
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



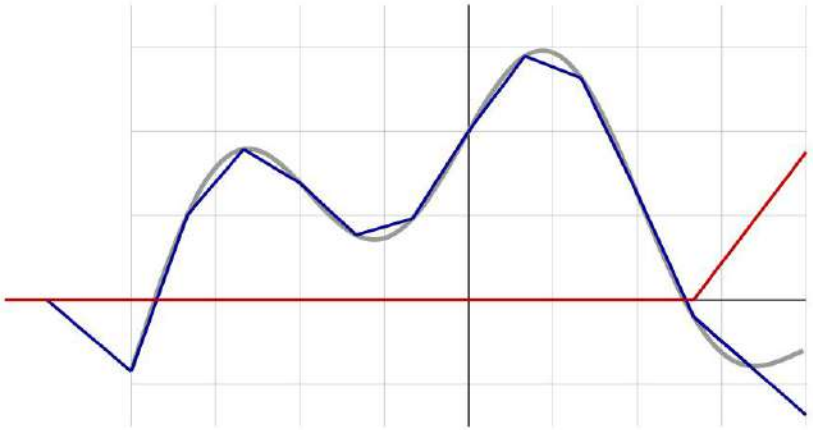
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



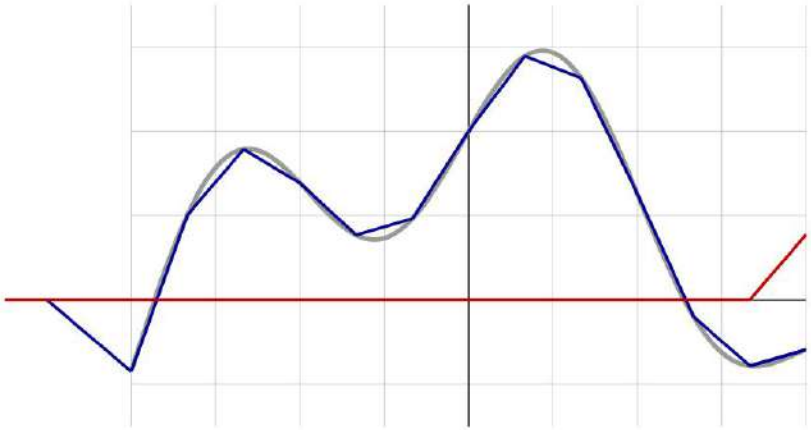
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

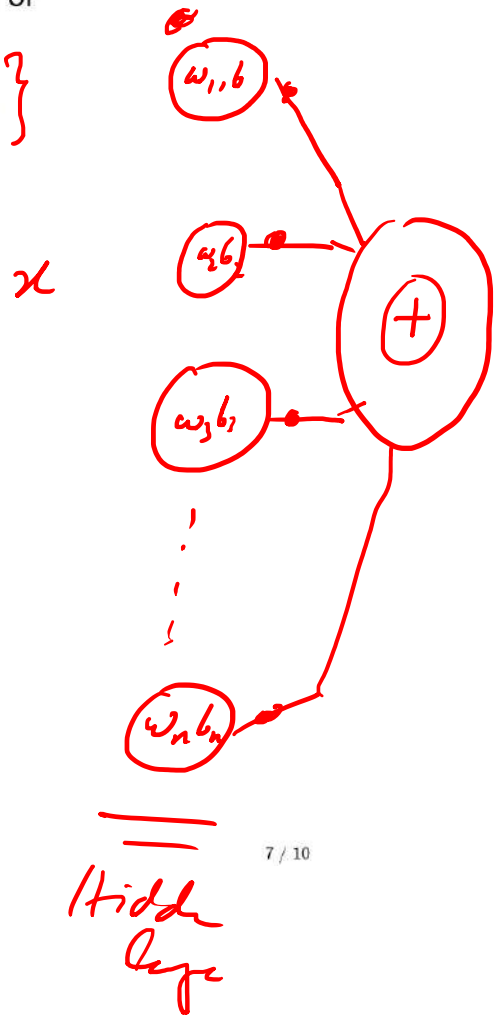
$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

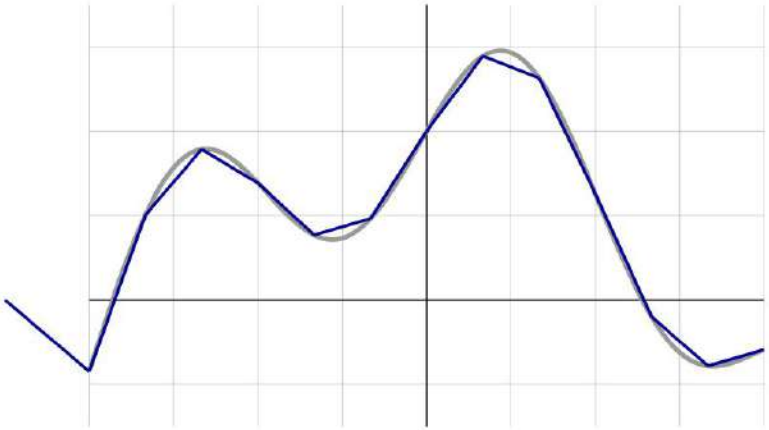
$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$





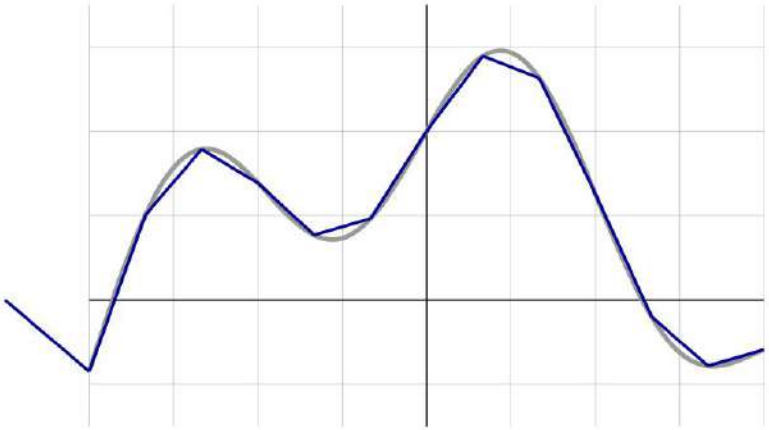
We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



We can approximate any $\psi \in \mathscr{C}([a, b], \mathbb{R})$ with a linear combination of translated/scaled ReLU functions.

$$f(x) = \sigma(w_1 x + b_1) + \sigma(w_2 x + b_2) + \sigma(w_3 x + b_3) + \dots$$



This is true for other activation functions under mild assumptions.

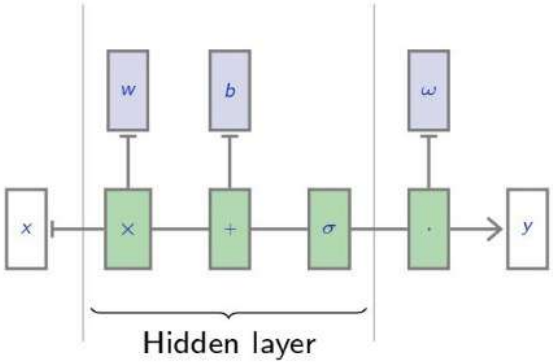
So we can approximate any continuous function

$$\psi : [0,1]^D \to \mathbb{R}$$

with a one hidden layer perceptron

$$x \mapsto \omega \cdot \sigma(w\,x + b),$$

where $b \in \mathbb{R}^K$, $w \in \mathbb{R}^{K \times D}$, and $\omega \in \mathbb{R}^K$.



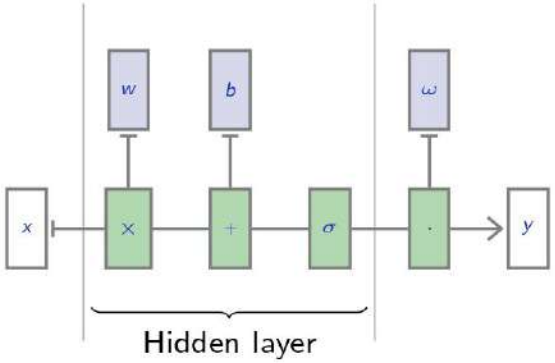
So we can approximate any continuous function

$$\psi : [0, 1]^D \to \mathbb{R}$$

with a one hidden layer perceptron

$$x \mapsto \omega \cdot \sigma(w x + b),$$

where $b \in \mathbb{R}^K$, $w \in \mathbb{R}^{K \times D}$, and $\omega \in \mathbb{R}^K$.



This is the **universal approximation theorem.**



A better approximation requires a larger hidden layer (larger K), and this theorem says nothing about the relation between the two.

So this results states that we can make the **training error** as low as we want by using a larger hidden layer. It states nothing about the **test error**.



A better approximation requires a larger hidden layer (larger K), and this theorem says nothing about the relation between the two.

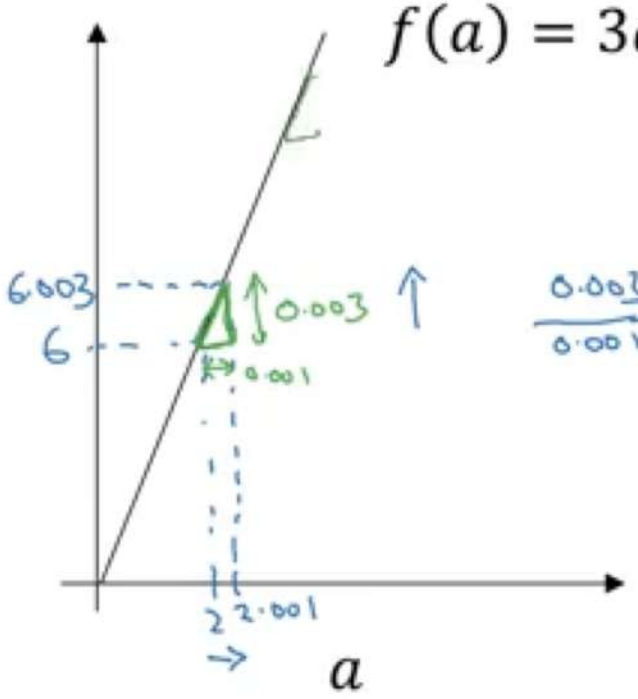
So this results states that we can make the **training error** as low as we want by using a larger hidden layer. It states nothing about the **test error**.

Deploying MLP in practice is often a balancing act between under-fitting and over-fitting.

Optimization Problem

HOW TO DETERMINE WEIGHTS?

Intuition about derivatives



$\frac{0.003}{0.001}$ $\frac{\text{height}}{\text{width}}$

→ slope (derivative) of $f(a)$ at $a=2$ is 3

→ $a = 2$ $f(a) = 6$

$a = 2.001$ $f(a) = 6.003$

→ $a = 5$ $f(a) = 15$

$a = 5.001$ $f(a) = 15.003$

slope at $a=5$ is also 3

$\frac{d f(a)}{da} = 3 = \frac{d}{da} f(a)$

0.001 ←

0.00000001

0.000000001

Andrew Ng

The **basic definition of the slope of a function** is:

👉 The slope tells us **how steep a function is** at a certain point, i.e., how much the function's output (**y**) changes when the input (**x**) changes.

Formally:

- For a **straight line** $y = mx + c$, the slope is the constant m, meaning:

$$\text{slope} = \frac{\Delta y}{\Delta x}$$

  (the change in y divided by the change in x).
- For a **curve** $y = f(x)$, the slope at a point is the **derivative**:

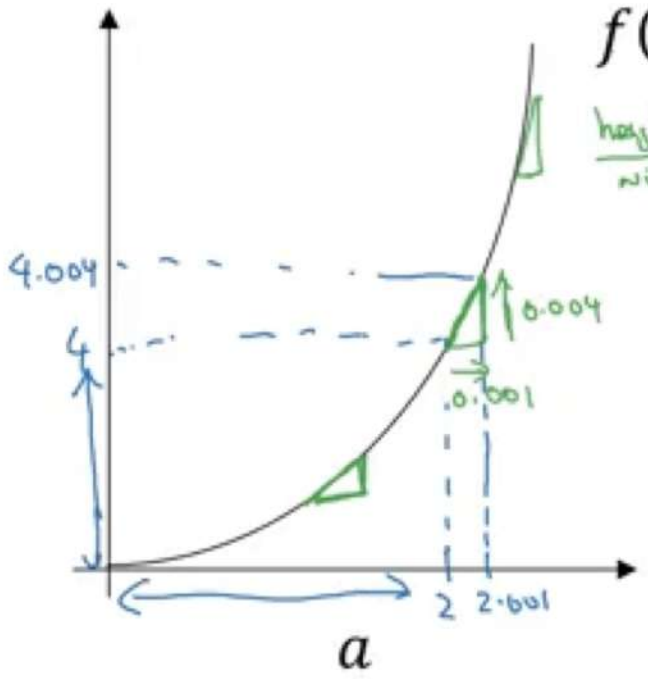
$$\text{slope at } x = f'(x)$$

  which gives the rate of change of y with respect to x.

📌 So in simple words:

The slope of a function shows how fast the functio[illegible] going up or down when you move along the x-axis.

Intuition about derivatives



$\frac{d}{da} a^2 = 2a$

$0.001 \times (2a)$

0.001 ←
0.000000....01 ←

$a = 2$ $\quad f(a) = 4$

$a = 2.001$ $\quad f(a) \approx 4.004$ $\quad (4.004\boxed{001})$

slope (derivative) of $f(a)$ at $a = 2$ is 4.

$\frac{d}{da} f(a) = 4$ when $a = 2$

$a = 5$ $\quad f(a) = 25$

$a = 5.001$ $\quad f(a) \approx 25.010$

$\frac{d}{da} f(a) = 10$ when $a = 5$

$\frac{d}{da} f(a) = \frac{d}{da} a^2 = 2a$

Andrew Ng

More derivative examples

$f(a) = a^2$ $\quad \frac{d}{da} f(a) = \underbrace{2a}_{4}$

$a = 2$ $\quad f(a) = 4$

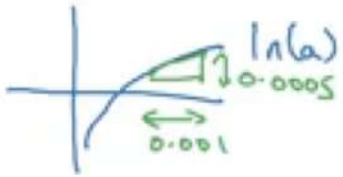
$a = 2.001$ $\quad f(a) \approx 4.004$

$f(a) = a^3$ $\quad \frac{d}{da} f(a) = \underbrace{3a^2}_{3 \times 2^2 = 12}$

$a = 2$ $\quad f(a) = 8$

$a = 2.001$ $\quad f(a) \approx 8.012$

$f(a) = \log_e(a)$ $\quad \ln(a)$ $\quad \frac{d}{da} f(a) = \frac{1}{a}$



$\frac{d}{da} f(a) = \boxed{\frac{1}{2}}$

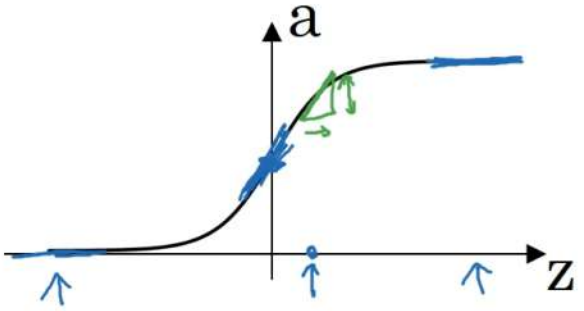
$a = 2$ $\quad f(a) \approx 0.69315$

$a = 2.001$ $\quad f(a) \approx 0.69265$

0.0005 $\quad 0.0005$

Andrew Ng

Sigmoid activation function



$$g(z) = \frac{1}{1+e^{-z}}$$

$a = g(z) = \frac{1}{1+e^{-z}}$

$g'(z) = \frac{d}{dz} g(z)$ = slope of $g(x)$ at z

$= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}}\right)$

$= g(z)\left(1 - g(z)\right)$ ←

$= a(1-a)$ | $g'(z) = a(1-a)$ ↑

$z = 10$. $g(z) \approx 1$

$\frac{d}{dz} g(z) \approx 1\,(1-1) \approx 0$

$z = -10$ $g(z) \approx 0$

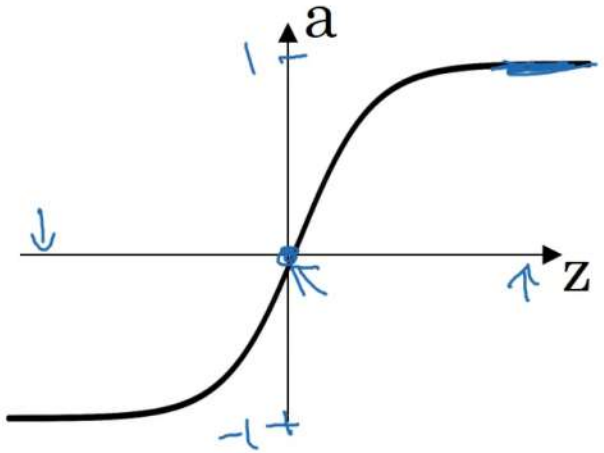
$\frac{d}{dz} g(z) \approx 0 \cdot (1-0) \approx 0$

$z = 0$ $g(z) = \frac{1}{2}$

$\frac{d}{dz} g(z) = \frac{1}{2}\left(1 - \frac{1}{2}\right) = \frac{1}{4}$

Andrew Ng

Tanh activation function



$$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

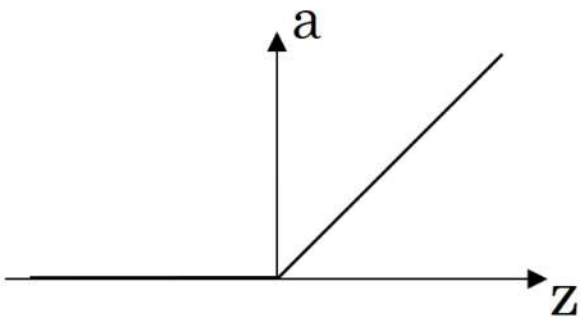
$$g'(z) = \frac{d}{dz} g(z) = \text{slope of } g(z) \text{ at } z = 1 - (\tanh(z))^2$$

$a = g(z)$, $g'(z) = 1 - a^2$

$z = 10$	$\tanh(z) \approx 1$, $g'(z) \approx 0$
$z = -10$	$\tanh(z) \approx -1$, $g'(z) \approx 0$
$z = 0$	$\tanh(z) = 0$, $g'(z) = 1$

Andrew Ng

ReLU and Leaky ReLU



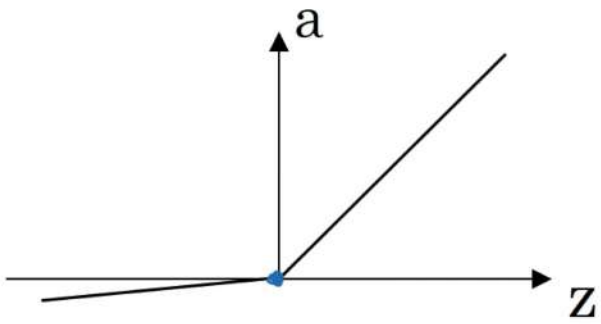
ReLU

$g(z) = \max(0, z)$

$g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$

~~undef if z=0~~

$z = 0.0000\ldots0$



Leaky ReLU

$g(z) = \max(0.01z, z)$

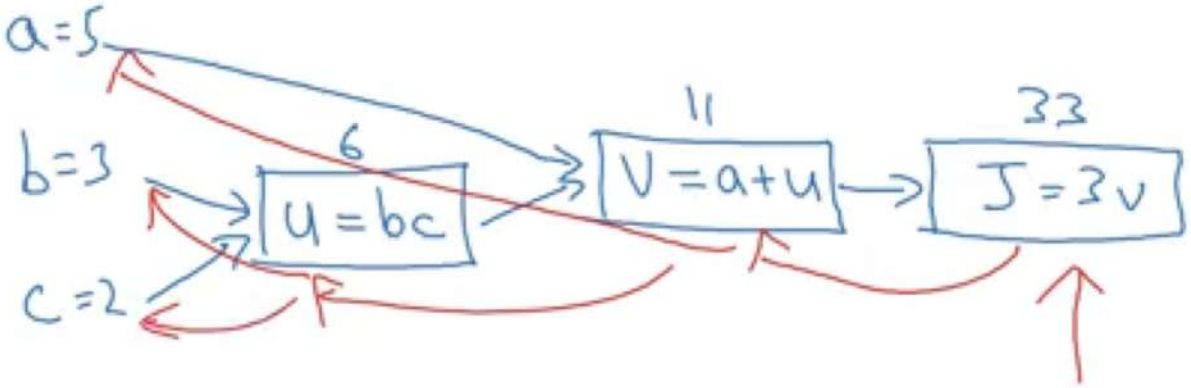
$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$

Andrew Ng

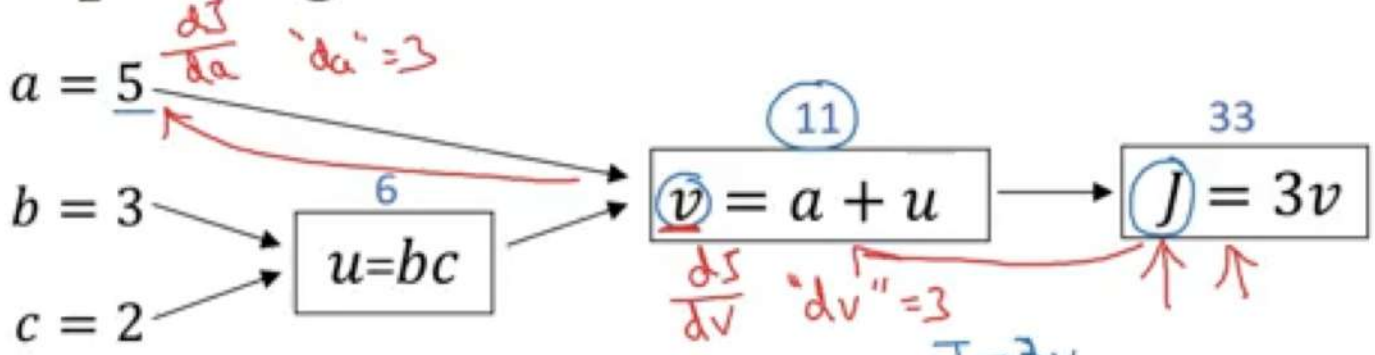
Computation Graph

$J(a,b,c) = 3(a + bc) = 3(5 + 3 \times 2) = 33$

u = bc, v = $a + u$, J = $3v$



Computing derivatives



$\frac{dJ}{dv} = ? = 3$

$\frac{dJ}{da} = 3 = \frac{dJ}{dv}\frac{dv}{da}$

3×1

$\frac{dv}{da} = 1$

$a \to v \to J$

$J = 3v$

$v = 11 \to 11.001$

$J = 33 \to 33.003$

$a = 5 \to 5.001$

$\to v = 11 \to 11.001$

$J = 33 \to 33.003$

$\frac{d\,\text{FinalOutputVar}}{d\,\text{var}}$ $\frac{dJ}{dvar}$ "dvar"

$f(a) = 3a$

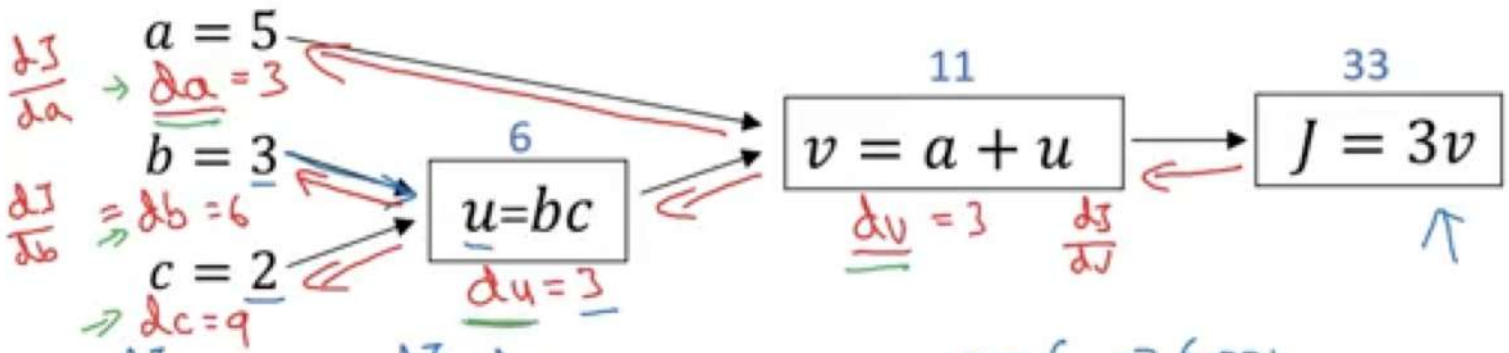
$\frac{df(a)}{da} = \frac{df}{da} = 3$

$J = 3v$

$\frac{dJ}{dv} = 3$

Andrew Ng

Computing derivatives



$\frac{dJ}{du} = 3 = \frac{dJ}{dv} \cdot \frac{dv}{du}$ (3 · 1)

$\frac{dJ}{db} = \frac{dJ}{du} \cdot \frac{du}{db} = 6$ (3 · 2)

$\frac{dJ}{dc} = \frac{dJ}{du} \cdot \frac{du}{dc} = 9$ (3 × 3)

$u = 6 \rightarrow 6.001$

$v = 11 \rightarrow 11.001$

$J = 33 \rightarrow 33.003$

$b = 3 \rightarrow 3.001$

$u = b \cdot c = 6 \rightarrow 6.002$

$J = 33.006$

$c = 2$

$.006$

$v = 11.002$

$J = 3v$

Andrew Ng

Gradient descent for neural networks

Parameters: $W^{[1]}$, $b^{[1]}$, $W^{[2]}$, $b^{[2]}$

$(n^{[1]}, n^{[0]})$ $(n^{[1]}, 1)$ $(n^{[2]}, n^{[1]})$ $(n^{[2]}, 1)$

$n_x = n^{[0]}$, $n^{[1]}$, $n^{[2]} = 1$

Cost function: $J(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}, y)$

($W^{[2]}$, $b^{[2]}$; $\hat{y} = a^{[2]}$)

Gradient descent:

→ Repeat {

→ Compute predicts $(\hat{y}^{(i)}, i = 1, \dots, m)$

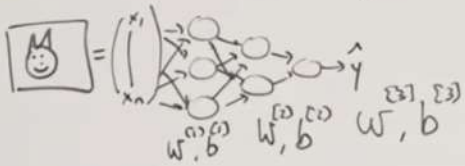
$dW^{[1]} = \frac{\partial J}{\partial W^{[1]}}$, $db^{[1]} = \frac{\partial J}{\partial b^{[1]}}$, ...

$W^{[1]} := W^{[1]} - \alpha \, dW^{[1]}$

$b^{[1]} := b^{[1]} - \alpha \, db^{[1]}$

Backpropagation

cost function: $J(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}^{(i)}(\hat{y}, y)$



with: $\mathcal{L}^{(i)} = -\left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$

Update: $w^{[\ell]} = w^{[\ell]} - \alpha \frac{\partial J}{\partial w^{[\ell]}}$

$\frac{\partial J}{\partial w^{[3]}}$ $\frac{\partial \mathcal{L}}{\partial w^{[3]}} = -\left[y^{(i)} \frac{\partial}{\partial w^{[3]}} \right($

Stanford

The derivation and discussion about vanishing and exploding gradients can be found here (please watch them carefully, from start to end)
:
https://www.youtube.com/watch?v=MfIjxPh6Pys&t=2154s
https://www.youtube.com/watch?v=zUazLXZZA2U

$$f(w) = \sigma(wa + b)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the **sigmoid function**.

Step 1: Define inside term

$$z = wa + b$$

So,

$$f(w) = \sigma(z)$$

Step 2: Derivative of sigmoid

We know a standard result:

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

Step 3: Chain rule

$$\frac{df}{dw} = \frac{d\sigma(z)}{dz} \cdot \frac{dz}{dw}$$

$$= \sigma(z)(1 - \sigma(z)) \cdot \frac{d}{dw}(wa + b)$$

Step 4: Differentiate inside

$$\frac{d}{dw}(wa + b) = a$$

Final Answer

$$\frac{d}{dw}\,\sigma(wa + b) = a \cdot \sigma(wa + b)\,(1 - \sigma(wa + b))$$